

GuardianClima ITBA

Documento del Proyecto
Challenge Tecnológico Integrador

Grupo: BDefenders

Cocce Maia - Gongora Rosi Inaki - Maccari Agustina - Sanchez Claria Facundo

ITBA - Instituto Tecnológico de Buenos Aires

Indice

1. Introduccion

2. Diseno y Arquitectura

2.1 Organizacion del proyecto

2.2 Flujo de la aplicacion

2.3 Decisiones de diseno

3. Guia de Usuario

3.1 Menu de Acceso

3.2 Menu Principal

4. Desafios y Soluciones

4.1 Mostrar la informacion de forma clara en la terminal

4.2 Ajuste de los estilos de la interfaz

5. Conclusiones y Aprendizajes del Equipo

1. Introduccion

GuardianClima ITBA es una aplicacion desarrollada en Python que permite consultar el clima de cualquier ciudad del mundo, mantener un historial de consultas, generar estadisticas de uso y recibir consejos de vestimenta generados por Inteligencia Artificial. Toda la experiencia ocurre dentro de la terminal, con una interfaz visual ordenada y moderna.

El objetivo del proyecto es integrar de forma practica varios conceptos vistos durante la cursada: programacion modular en Python, validacion de contraseñas (ciberseguridad), analisis de datos a partir de archivos CSV, uso de Inteligencia Artificial generativa y conexion con servicios en la nube mediante APIs.

La aplicacion arranca con un menu de acceso donde el usuario puede iniciar sesion o registrarse. Una vez autenticado, accede al menu principal con todas las funcionalidades. Los datos se guardan en archivos CSV, que tambien sirven como fuente para generar graficos en herramientas externas como Excel.

2. Diseno y Arquitectura

La aplicacion esta organizada en modulos con responsabilidades bien separadas. Esta estructura facilita el mantenimiento, evita repetir codigo y permite entender cada parte de forma independiente.

2.1 Organizacion del proyecto

- **Autenticacion (auth):** gestiona el registro, inicio de sesion y validacion de contraseñas segun criterios de seguridad.
- **Clima (clima):** se conecta con la API de OpenWeatherMap para traer los datos del clima y administra el historial de consultas en CSV.
- **Inteligencia Artificial (ia):** se comunica con Google Gemini para generar los consejos de vestimenta.
- **Estadisticas (estadisticas):** procesa el historial global para calcular estadisticas de uso.
- **Interfaz (ui):** contiene toda la parte visual: pantallas, menu y estilos construidos con la libreria Textual.
- **Datos (data):** almacena los archivos CSV de usuarios y del historial global de consultas.

2.2 Flujo de la aplicacion

El usuario pasa primero por el menu de acceso (login o registro). Solo despues de autenticarse accede al menu principal:

Opcion	Que hace
1. Consultar Clima	Busca una ciudad y muestra temperatura, humedad, viento y descripcion del clima. La c
2. Mi Historial	Muestra en tabla todas las consultas anteriores del usuario, con filtro por ciudad.
3. Estadisticas	Calcula ciudad mas consultada, total de consultas y temperatura promedio global.
4. Consejo IA	Genera un consejo de vestimenta con IA usando la ultima consulta, una del historial o u
5. Acerca De	Muestra informacion de la aplicacion y del equipo.
6. Cerrar Sesion	Finaliza la sesion y regresa al menu de acceso.

2.3 Decisiones de disenio

La principal decision de disenio fue utilizar la libreria Textual para construir la interfaz, en lugar de texto plano de consola. Esto permitio mostrar la informacion en paneles, tablas y menus navegables, logrando una experiencia mucho mas clara. Ademas, las claves de las APIs se almacenan en un archivo .env por seguridad, y los datos se guardan en CSV para reutilizarlos facilmente en Excel.

3. Guia de Usuario

3.1 Menu de Acceso

- **Iniciar Sesion:** el usuario ingresa su nombre y contrasena. Si los datos son correctos, entra directamente al menu principal.
- **Registrarse:** el usuario elige un nombre y una contrasena. Esta debe cumplir cinco criterios de seguridad: longitud minima, mayuscula, minuscula, numero y caracter especial. Si no los cumple, el sistema indica que falta y sugiere como mejorarla. Tras un registro exitoso, el usuario ingresa automaticamente.
- **Salir:** cierra la aplicacion.

3.2 Menu Principal

- **Consultar Clima:** se escribe el nombre de una ciudad (con sugerencias automaticas) y se muestran temperatura, sensacion termica, humedad, viento y descripcion del clima. La consulta queda guardada en el historial.
- **Mi Historial:** muestra en tabla todas las consultas anteriores del usuario, con opcion de filtrar por ciudad.
- **Estadisticas:** presenta la ciudad mas consultada, el total de consultas y la temperatura promedio, junto con graficos de barras en texto.

- **Consejo IA:** ofrece un consejo de vestimenta generado por IA. Se puede usar la ultima consulta, elegir una entrada del historial o realizar una nueva busqueda especifica.
- **Acerca De:** muestra informacion de la aplicacion y del equipo.
- **Cerrar Sesion:** finaliza la sesion y regresa al menu de acceso.

4. Desafios y Soluciones

4.1 Mostrar la informacion de forma clara en la terminal

El primer gran desafio fue presentar toda la informacion de manera clara y ordenada usando solamente la terminal. Con texto plano era dificil organizar los datos del clima, las tablas del historial y los menus de navegacion de forma legible.

De este problema surgio la idea de utilizar Textual, una libreria de Python que permite crear interfaces visuales completas dentro de la terminal. Gracias a Textual pudimos mostrar la informacion en paneles, tablas y menus, mejorando enormemente la claridad y la experiencia de uso.

4.2 Ajuste de los estilos de la interfaz

Una vez adoptado Textual, aparecio un segundo desafio: el manejo de los estilos. Al principio se mostraban cosas raras, con espacios y elementos que aparecian en lugares inesperados. Resolver estos problemas requirio dedicarle tiempo, hacer pruebas iterativas y aplicar conocimientos de prompt engineering para encontrar la causa raiz y corregir el comportamiento de la interfaz hasta lograr que se viera prolija y ordenada.

5. Conclusiones y Aprendizajes del Equipo

El aprendizaje mas importante que nos llevamos como equipo es la relevancia de ser ordenados y trabajar con modularidad al programar. A lo largo del proyecto comprobamos lo util que es separar el codigo en modulos y funciones bien definidas.

Un ejemplo concreto visible en nuestro repositorio: existen funciones que verifican si los archivos necesarios estan creados y, si no existen, los crean automaticamente. Esas funciones se reutilizan en muchas otras partes del codigo, que siempre revisan la existencia del archivo antes de trabajar con el. Esto nos demuestra que organizar bien las carpetas y centralizar este tipo de verificaciones evita repetir codigo y hace que el proyecto sea mucho mas entendible y facil de mantener.

En resumen, el challenge nos permitio integrar conocimientos de programacion, ciberseguridad, analisis de datos, inteligencia artificial y conectividad en la nube, y al mismo tiempo reforzar la importancia del orden y la buena estructura del codigo como bases de todo proyecto de software.